

PROGRAMMING PRACTICE / EXPERIMENT 02

在线评测系统

Atelier OJ

FASTAPI

STREAMLIT

LINUX RUNNER

AI AUTHORIZING

一个具有真实异步评测、权限审计与可验证 AI 命题流程的课程级 Online Judge。

姓名 _____

学号 _____

班级 _____

GitHub [jasonpiggg / Online-Judge-System](#)

2026 · PYTHON 3.12 · UBUNTU / WSL2

1. 实验目标与完成情况

本实验从空仓库实现一个可运行、可测试、可追踪开发过程的小型 Online Judge。系统不仅完成题目管理、用户管理和提交查询，还真正执行 Python 与 C++14 程序，逐测试点给出结果、用时和内存；在此基础上实现日志公开与访问审计，并扩展了流式 AI 智能命题工作台。

评分模块	分值	完成情况
Step 1 题目管理	5	完整 CRUD、JSON 配置、校验、权限与初始题目
Step 2 评测控制	5	编译、运行、七类结果、时间/内存/输出限制
Step 3 提交管理	5	异步任务、恢复、筛选分页、限频、重测
Step 4 用户管理	5	注册登录、Session、角色、禁用和统计
Step 5 日志与审计	5	私有/公开策略、200/403 审计与筛选
Step 6 Web 界面	5	面向用户、管理员及 AI 的完整 Streamlit 页面
Advance AI 命题	10	流式生成、批判改进、真实取消、验证与计费
工程规范	5	分层结构、类型/格式/测试、CI、真实 PR 流程
实验报告	5	设计、实现、测试、安全边界和开发复盘

2. 系统设计

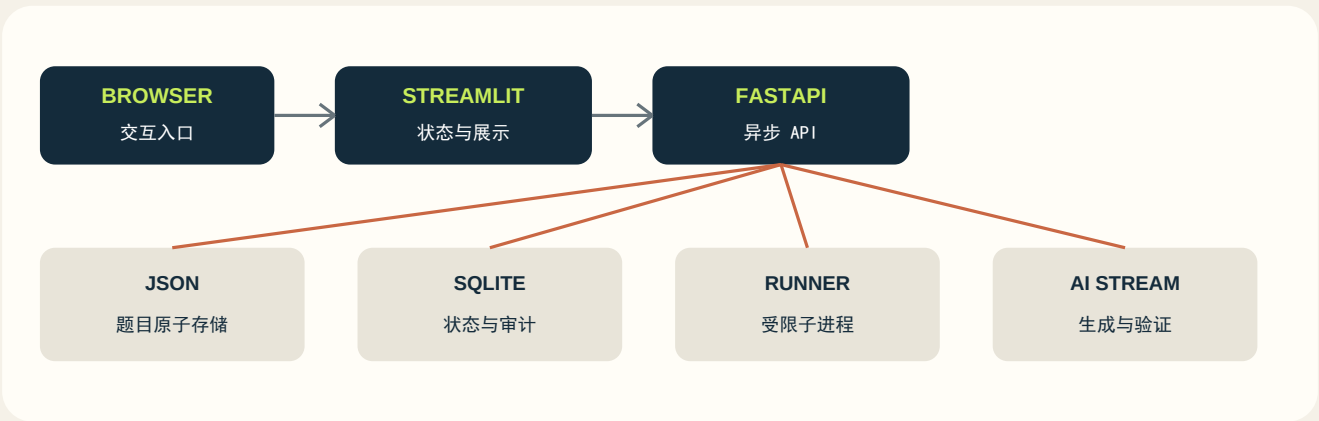


图 1 系统组件与数据流

2.1 总体架构

系统采用前后端分离方式。Streamlit 只负责交互与展示，所有业务操作均经 REST API 完成；FastAPI 负责参数校验、身份鉴别、权限判断和业务编排；SQLite 保存结构化运行数据，题目则按要求以独立 JSON 文件保存。评测与 AI 生成均通过 `asyncio.create_task` 在后台运行，创建请求可立即返回，前端通过 `fragment` 轮询最终状态。

```
浏览器 → Streamlit → FastAPI
├─ ProblemStore → JSON / 原子替换
├─ aiosqlite → 用户、Session、提交、日志、AI 任务
├─ JudgeEngine → 临时目录 / subprocess / rlimit
└─ AIAuthoring → 流式兼容 API / 本地参考解验证
```

2.2 数据持久化

题目 ID 只接受字母、数字、下划线和连字符，文件名不能构造目录穿越。新增和修改先通过 Pydantic 完整校验，再写入同目录临时文件，`fsync` 后原子替换正式 JSON；异步锁保证同一进程内并发写入顺序。版本库只保存 `data/problem_seeds/`，启动时复制到 `var/problems/`，因此 `POST /api/reset/` 能确定性恢复初始状态。

SQLite 中分别保存 `users`、`sessions`、`languages`、`submissions`、`submission_cases`、`access_logs`、`ai_model_configs` 和 `ai_problem_tasks`。数据库启用外键、WAL 与必要索引。接口层不返回密码哈希、加密密钥、服务端路径或原始异常。

3. 关键功能实现

3.1 用户、Session 与权限

密码使用 bcrypt 单向哈希。登录成功后轮换高熵 Session ID, Cookie 设置 HttpOnly、SameSite 和过期时间; 服务端数据库保存 Session, 因此退出可立即失效。启动时创建课程指定的 admin / admintestpassword。角色包括 user、admin、banned, 禁用用户无论已有会话还是重新登录均返回 403。

用户详情中的提交数与通过题数实时聚合, 后者按 problem_id 去重。公共依赖把鉴权检查放在业务参数检查之前, 遵守 401、403、400、429、409、404、500 的错误优先级。Pydantic 的默认 422 由异常处理器转换为课程要求的 400, 响应统一包含 code、msg、data。

3.2 题目管理

题目模型覆盖题号、标题、描述、输入输出说明、约束、提示、标签、时间和内存限制、样例、隐藏测试点及 public_cases。题号重复返回 409, 不存在返回 404; 更新要求 URL 与请求体题号一致。所有读写均要求登录, 删除和日志可见性修改仅管理员执行。

初始题目“两数之和”可直接演示。题目列表返回适合卡片展示的摘要, 详情返回完整结构; 编辑器可从已有题目或 AI 结果载入, 人工确认后仍调用标准新增/更新接口, 不绕过校验。

3.3 语言配置与评测器

系统默认注册 Python 3 与 C++14。命令模板只允许 {src}、{exe} 占位符, 经 shlex 展开为 argv; 拒绝管道、重定向、命令替换和 shell 执行。每次评测使用独立临时目录与最小环境变量, Python 直接执行, C++ 先编译; 编译失败返回 CE 和裁剪后的结构化 compile_info。

Linux runner 使用异步 subprocess。子进程在独立进程组运行, 墙钟超时后终止整个进程组; psutil 周期采集进程树 RSS, 结合 RLIMIT_AS 控制内存; stdout/stderr 均有大小上限。每个测试点记录 AC、WA、TLE、MLE、RE、CE 或 UNK、用时、峰值内存和安全裁剪的错误信息。输出比较只忽略行末空白和最终多余换行, 其他字符严格一致。每个 AC 测试点计 10 分, 总分上限为测试点数乘 10。

3.4 提交、限频和恢复

POST /api/submissions/ 先持久化 pending 状态, 再创建后台任务, 因此接口不会阻塞到代码执行结束。应用启动时扫描遗留 pending/running 记录并重新调度, 避免服务重启产生永久悬挂任务。正常判题完成的 submission 状态为 success, 评测基础设施异常才记为 error。

每个用户使用数据库时间窗口限制一分钟最多三次提交, 第 4 次返回 429。列表严格支持用户、题目、状态的一级/二级组合筛选和课程规定的分页语义; 普通用户只能看到本人结果, 管理员可查看全站并发起 rejudge。重测会清理旧测试点、覆盖总体状态并重新进入 pending。

3.5 测试点日志与访问审计

提交本人和管理员可查看逐点日志; 管理员将题目设为 public_cases=true 后, 其他已登录用户也可查看该题目的测试点结果, 但仍不能访问提交者私有概要。对存在的 submission, 日志访问成功与权限拒绝分别写入 200/403 审计; 未登录、参数格式错误和不存在的 submission 不记录, 避免无意义日志与枚举信息。管理员可按用户、题目和分页查询审计记录。

3.6 Streamlit 交互界面



图 2 639 px 窄屏浏览器验收界面结构化重绘

界面采用 editorial laboratory 视觉：暖灰纸张背景、深墨蓝、少量酸橙强调、细网格纹理和紧凑等宽状态标签。布局不依赖模拟数据，requests.Session 保存在 st.session_state 中传递后端 Cookie。页面覆盖注册登录、个人资料、题目浏览和编辑、Ace 代码编辑器、提交轮询、结果/编译/逐点日志、用户角色、语言、审计及日志公开设置。

浏览器验收以管理员身份完成登录和题目详情 workflow；在 639 px 窄屏视口中确认侧栏、卡片、标签页和操作按钮可用，控制台无错误。pending submission 与 AI task 使用 Streamlit fragment 定时刷新，任务结束后停止轮询。

3.7 AI 智能命题

每位用户可独立设置 provider URL、模型、API key、输入/输出单价和计价单位。API key 由环境主密钥派生的 Fernet 加密后入库，查询接口只暴露是否已配置。provider 默认要求 HTTPS 和公网地址，防止利用服务器访问环回或内网；仅自动测试可显式允许本地 mock。

任务调用 OpenAI-compatible chat/completions 流式接口，持续保存需求分析、题目生成、测试点审查和结果校验状态。取消接口会取消实际 asyncio Task，并由上下文关闭 HTTP 流；已结束任务再次取消返回 409。生成阶段先输出完整结构，第二阶段批判知识点、难度、边界覆盖和潜在错误解法并改进，随后把参考解交给同一个受限评测器运行全部测试点。任何 schema 或本地验证失败都不会标记为可入库。

Token 优先采用服务商 usage；缺失时保存明确标注的估算值。费用按输入/输出 token、各自单价和计价单位分别计算后求和，前端同步显示依据。通过验证的结果只能载入编辑器，最终保存仍需人工审阅。

4. 测试与验收

测试由 `pytest`、`pytest-asyncio`、`HTTPX` 和 `Streamlit AppTest` 组成。单元/API 测试覆盖模型、原子存储、密码与 `Session`、权限矩阵、分页、输出规范化、计分、命令模板、限频、重测、审计、密钥脱敏、Token 费用、AI 流式协议和取消。Linux 专项用例实际调用 Python 与 `g++`，验证 Python TLE、C++ AC 和 CE；Windows 本地运行时自动跳过这些仅 Linux 才有完整语义的用例。

最终质量门禁：

```
ruff check src frontend tests
mypy src
pytest --cov=src/oj --cov-report=term-missing --cov-fail-under=85
```

验收环境	结果	覆盖率
Windows / Python 3.14 (界面开发)	22 passed, 2 Linux-only skipped	85.65%
GitHub Actions Ubuntu / Python 3.12	24 passed	87.63%

GitHub Actions 固定 Ubuntu 与 Python 3.12，使用系统 `g++`，所有 Pull Request 必须在 lint、类型检查和覆盖率测试通过后合并。端到端验收链路为：注册 → 登录 → 浏览题目 → 提交 Python/C++ → 轮询 → 查看日志 → 管理员公开日志/重测/修改角色 → AI 生成 → 审阅并保存题目。

5. Edge Cases、安全与性能

- 题号、分页、源代码、编译输出和运行输出均限制格式或大小；错误响应不泄露内部路径。
- 普通用户无法通过筛选参数越权查看他人提交；公开日志不等于公开提交概要。
- 后台任务集中登记，重测和取消具有确定状态迁移；单 Unicorn worker 保证进程内任务表语义。
- SQLite WAL 和短事务适合课程规模；更大并发应迁移 PostgreSQL、持久任务队列和对象存储。
- rlimit + psutil + ■■■■ 是实验级防护，不能替代容器/虚拟机、cgroup、seccomp、只读根文件系统和网络隔离。
- 服务默认仅绑定 localhost。未引入生产级沙箱前，不应将任意代码执行端点部署到公网。

6. 工程过程与 AI 工具说明

仓库从空目录建立，按 `feat/auth-users`、`feat/problem-management`、`feat/judge-engine`、`feat/submission-management`、`feat/audit-logs`、`feat/streamlit-frontend`、`feat/ai-problem-authoring` 和 `test/docs-release` 八个短期分支推进。每阶段使用 Conventional Commits，推送后创建真实 PR，以 `merge commit` 合入 `main` 并删除分支；提交历史、CI 证据和修复记录均保留。

本项目使用 OpenAI Codex 辅助需求拆分、代码实现、测试设计、浏览器验收和文档生成。按新增代码的初始产出估算，约 90% 由 AI 辅助生成，约 10% 为需求约束、取舍与验收反馈；所有功能以自动化测试、静态检查、真实 CI 和运行结果为准，而不是把模型输出本身视为正确性证据。AI 服务密钥、数据库和运行产物均未提交到仓库。

7. 总结与改进方向

实验完成了从数据建模、异步 Web API、权限系统到不可信代码执行控制的完整工程闭环，并把可观测的 AI 工作流纳入同一套权限和持久化模型。最关键的收获是：Online Judge 的难点并非启动一个子进程，而是把状态一致性、超时清理、资源限制、可见性规则和可重复测试同时做对。

后续可把评测任务迁移到隔离容器和持久队列，增加 WebSocket 推送、比赛榜单、题目版本、判题机横向扩展及 PostgreSQL；AI 命题可加入多模型交叉审查、变异测试和标准解复杂度静态分析。

课程文档：<<https://dbg-course.github.io/python-docs/oj/>>